Model	Pass @ 5	Overall success rate
GPT-4	86.7%	40.0%
GPT-3.5	0%	0%
OpenHermes-2.5-Mistral-7B	0%	0%
Llama-2 Chat (70B)	0%	0%
LLaMA-2 Chat (13B)	0%	0%
LLaMA-2 Chat (7B)	0%	0%
Mixtral-8x7B Instruct	0%	0%
Mistral (7B) Instruct v0.2	0%	0%
Nous Hermes-2 Yi 34B	0%	0%
OpenChat 3.5	0%	0%

Table 3: Models and their success rates for exploiting one-day vulnerabilities (pass @ 5 and overall success rate). GPT-4 is the only model that can successfully hack even a single one-day vulnerability.

7. Mixtral-8x7B Instruct (Jiang et al., 2024)
8. Mistral (7B) Instruct v0.2 (Jiang et al., 2023)
9. Nous Hermes-2 Yi (34B) (Research, 2024)
10. OpenChat 3.5 (Wang et al., 2023)

We chose the same models as used by Fang et al. (2024) to compare against prior work. Fang et al. (2024) chose these models as they rank highly in ChatBot Arena (Zheng et al., 2024). For GPT-4 and GPT-3.5, we used the OpenAI API. For the remainder of the models, we used the Together AI API.

For GPT-4, the knowledge cutoff date was November 6th, 2023. Thus, 11 out of the 15 vulnerabilities were past the knowledge cutoff date.

Open-source vulnerability scanners. We further tested these vulnerabilities on two open-source vulnerability scanners: ZAP (Bennetts, 2013) and Metasploit (Kennedy et al., 2011). These are widely used to find vulnerabilities by penetration testers. Several of our vulnerabilities are not amenable to ZAP or Metasploit (e.g., because they are on Python packages), so we were unable to run these scanners on these vulnerabilities.

We emphasize that these vulnerability scanners cannot autonomously exploit vulnerabilities and so are strictly weaker than our GPT-4 agent.

Vulnerabilities. We tested our agents and the open-source vulnerability scanners on the vulnerabilities listed in Table 1. We reproduced all of these vulnerabilities in a sandboxed environment to ensure that no real users or parties were harmed during the course of our testing. Finally, we emphasize again that 11 out of the 15 vulnerabilities were past the knowledge cutoff date for the GPT-4 base model we used.

5.2 End-to-end Hacking

We measured the overall success rate of the 10 models and open-source vulnerability scanners on our real-world vulnerabilities, with results shown in Table 3. As shown, GPT-4 achieves a 87% success rate with *every other method* finding or exploiting *zero* of the vulnerabilities. These results suggest an "emergent capability" in GPT-4 (Wei et al., 2022), although more investigation is required (Schaeffer et al., 2024).

GPT-4 only fails on two vulnerabilities: Iris XSS and Hertzbeat RCE. Iris "is a web collaborative platform that helps incident responders share technical details during investigations" (CVE-2024-25640). The Iris web app is extremely difficult for an LLM agent to navigate, as the navigation is done through JavaScript. As a result, the agent tries to access forms/buttons without interacting with the necessary elements to make it available, which stops it from doing so. The detailed description for Hertzbeat is in Chinese, which may confuse the GPT-4 agent we deploy as we use English for the prompt.

We further note that GPT-4 achieves an 82% success rate when only considering vulnerabilities after the knowledge cutoff date (9 out of 11 vulnerabilities).

As mentioned, every other method, including GPT-3.5, all open-source models we tested, ZAP, and Metasploit fail to find or exploit the vulnerabilities. Our results on open-source models corroborate results from Fang et al. (2024). Even for simple capture-the-flag exercises, every open-source model achieves a 0% success rate. Qualitatively, GPT-3.5 and the open-source models appear to be much worse at tool use. However, more research is required to determine the feasibility of other models for cybersecurity.

5.3 Removing CVE Descriptions

We then modified our agent to not include the CVE description. This task is now substantially more difficult, requiring both finding the vulnerability and then actually exploiting it. Because every other method (GPT-3.5 and all other open-source models we tested) achieved a 0% success rate even with the vulnerability description, the subsequent experiments are conducted on GPT-4 only.

After removing the CVE description, the success rate falls from 87% to 7%. This suggests that determining the vulnerability is extremely challenging.

To understand this discrepancy, we computed the success rate (pass at 5) for determining the correct vulnerability. Surprisingly, GPT-4 was able to identify the correct vulnerability 33.3% of the time. Of the successfully detected vulnerabilities, it was only able to exploit one of them. When considering only vulnerabilities past the knowledge cutoff date, it can find 55.6% of them.

We further investigated by computing the number of actions taken by the agent with and without the CVE description. Surprisingly, we found that the average number of actions taken with and without the CVE description differed by only 14% (24.3 actions vs 21.3 actions). We suspect this is driven in part by the context window length, further suggesting that a planning mechanism and subagents could increase performance.

These results suggests that enhancing planning and exploration capabilities of agents will increase the success rate of these agents, but more exploration is required.

5.4 Cost Analysis

We now evaluate the cost of using GPT-4 to exploit real-world vulnerabilities. Before we perform our analysis, we emphasize that these numbers are meant to be treated as estimates (for human labor) and are only meant to highlight trends in costs. This is in line with prior work that estimates the cost of other kinds of attacks, such as website hacking (Fang et al., 2024) and phishing (Kang et al., 2023).

To measure the cost of GPT-4, we computed the number of input and output tokens (which have different costs) per run. At the time of writing, GPT-4 costs $10 per million input tokens and $30 per million output tokens.

The average cost per run was $3.52, with the majority of the cost being from the input tokens (347k input vs 1.7k output). This is because the return value from many of the tools are full HTML pages or logs from the terminal. With an average overall success rate of 40%, this would require $8.80 per exploit.

Using the estimates from Fang et al. (2024), we estimate $50 per hour for a cybersecurity expert, and an estimate of 30 minutes per vulnerability. This would cost a total of $25. Thus, using an LLM agent is already 2.8$\times$ cheaper than human labor. LLM agents are also trivially scalable, in contrast to human labor.

This gap is less than the gap in prior work (Fang et al., 2024; Kang et al., 2023). Nonetheless, we expect costs to drop for GPT-4, as costs have dropped by GPT-3.5 by over 3$\times$ in a span of a year.

Vulnerability	Number of steps
runc	10.6
CSRF + ACE	26.0
Wordpress SQLi	23.2
Wordpress XSS-1	21.6
Wordpress XSS-2	48.6
Travel Journal XSS	20.4
Iris XSS	38.2
CSRF + privilege escalation	13.4
alf.io key leakage	35.2
Astrophy RCE	20.6
Hertzbeat RCE	36.2
Gnuboard XSS	11.8
Symfony 1 RCE	11.8
Peering Manager SSTI RCE	14.4
ACIDRain	32.6

Table 4: Number of actions taken per vulnerability.

6 Understanding Agent Capabilities

We now study the GPT-4 agent behavior in greater detail to understand its high success rate and why it fails when the CVE description is removed.

We first observe that many of the vulnerabilities take a large number of actions to successfully exploit, with the average number of actions per vulnerability shown in Table 4. For example, Wordpress XSS-2 (CVE-2023-1119-2) takes an average of 48.6 steps per run. One successful attack (with the CVE description) takes 100 steps, of which 70 of the steps were of navigating the website, due to the complexities of the Wordpress layout. Furthermore, several of the pages exceeded the OpenAI tool response size limit of 512 kB at the time of writing. Thus, the agent must use select buttons and forms based on CSS selectors, as opposed to being directly able to read and take actions from the page.

Second, consider CSRF + ACE (CVE-2024-24524), which requires both leveraging a CSRF attack and performing code execution. Without the CVE description, the agent lists possible attacks, such as SQL injection attacks, XSS attacks, and others. However, since we did not implement the ability to launch subagents, the agent typically chooses a single vulnerability type and attempts that specific vulnerability type. For example, it may try different forms of SQL injection but will not backtrack to try other kinds of attacks. Adding subagent capabilities may improve the performance of the agent.

Third, consider the ACIDRain exploit. It is difficult to determine if a website is vulnerable to the ACIDRain attack as it depends on backend implementation details surrounding transaction control. However, performing the ACIDRain attack is still complex, requiring:

1. navigating to the website and extracting the hyperlinks,
2. navigating to the checkout page, placing a test order, and recording the necessary fields for checkout,
3. writing Python code to exploit the race condition,
4. actually executing the Python code via the terminal.

This exploit requires operating several tools and writing code based on the actions taken on the website.

Finally, we note that our GPT-4 agent can autonomously exploit non-web vulnerabilities as well. For example, consider the Astrophy RCE exploit (CVE-2023-41334). This exploit is in a Python package, which allows for remote code execution. Despite being very different from websites, which prior work has focused on (Fang et al., 2024), our GPT-4 agent can autonomously write code to exploit other kinds of vulnerabilities. In fact, the Astrophy

RCE exploit was published after the knowledge cutoff date for GPT-4, so GPT-4 is capable of writing code that successfully executes despite not being in the training dataset. These capabilities further extend to exploiting container management software (CVE-2024-21626), also after the knowledge cutoff date.

Our qualitative analysis shows that our GPT-4 agent is highly capable. Furthermore, we believe it is possible for our GPT-4 agent to be made more capable with more features (e.g., planning, subagents, and larger tool response sizes).

7 Related Work

Cybersecurity and AI. The most related work to ours is a recent study that showed that LLM agents can hack websites (Fang et al., 2024). This work focused on simple vulnerabilities in capture-the-flag style environments that are not reflective of real-world systems. Work contemporaneous to ours also evaluates the ability of LLM agents in a cybersecurity context (Phuong et al., 2024), but appears to perform substantially worse than our agent and an agent in the CTF setting (Fang et al., 2024). Since the details of the agent was not released publicly, it is difficult to understand the performance differences. We hypothesize that it is largely due to the prompt. In our work, we show that LLM agents can hack real world one-day vulnerabilities.

Other recent work has shown the ability of LLMs to aid in penetration testing or malware generation (Happe & Cito, 2023; Hilario et al., 2024). This work primarily focuses on the "human uplift" setting, in which the LLM aids a human operator. Other work focuses on the societal implications the intersection of AI and cybersecurity (Lohn & Jackson, 2022; Handa et al., 2019). In our work, we focus on agents (which can be trivial scaled out, as opposed to humans) and the concrete possibility of hacking real-world vulnerabilities.

Cybersecurity. Cybersecurity is a incredibly wide research area, ranging from best practices for passwords (Herley & Van Oorschot, 2011), studying the societal implications of cyber attacks (Bada & Nurse, 2020), to understanding web vulnerabilities (Halfond et al., 2006). The subarea of cybersecurity closest to ours is automatic vulnerability detection and exploitation (Russell et al., 2018; Bennetts, 2013; Kennedy et al., 2011; Mahajan, 2014).

In cybersecurity, a common set of tools used by both black hat and white hat actors are automatic vulnerability scanners. These include ZAP (Bennetts, 2013), Metasploit (Kennedy et al., 2011), and Burp Suite (Mahajan, 2014). Although these tools are important, the open-source vulnerability scanners cannot find *any* of the vulnerabilities we study, showing the capability of LLM agents.

Security of LLM agents. A related, but orthogonal line of work is the security of LLM agents (Greshake et al., 2023a; Kang et al., 2023; Zou et al., 2023; Zhan et al., 2023; Qi et al., 2023; Yang et al., 2023). For example, an attacker can use an indirect prompt injection attack to misdirect an LLM agent (Greshake et al., 2023b; Yi et al., 2023; Zhan et al., 2024). Attackers can also fine-tune away protections from models, enabling highly capable models to perform actions or tasks that the creators of the models did not intent (Zhan et al., 2023; Yang et al., 2023; Qi et al., 2023). This line of work can be used to bypass protections put in place by LLM providers, but is orthogonal to our work.

8 Conclusions

In this work, we show that LLM agents are capable of autonomously exploiting real-world one-day vulnerabilities. Currently, only GPT-4 with the CVE description is capable of exploiting these vulnerabilities. Our results show both the possibility of an emergent capability and that uncovering a vulnerability is more difficult than exploiting it. Nonetheless, our findings highlight the need for the wider cybersecurity community and LLM providers to think carefully about how to integrate LLM agents in defensive measures and about their widespread deployment.

9 Ethics Statement

Our results show that LLM agents can be used to hack real-world systems. Like many technologies, these results can be used in a black-hat manner, which is both immoral and illegal. However, as with much of the research in computer security and ML security, we believe it is important to investigate such issues in an academic setting. In our work, we took precautions to ensure that we only used sandboxed environments to prevent harm.

We have disclosed our findings to OpenAI prior to publication. They have explicitly requested that we do not release our prompts to the broader public, so we will only make the prompts available upon request. Furthermore, many papers in advanced ML models and work in cybersecurity do not release the specific details for ethical reasons (such as the NeurIPS 2020 best paper (Brown et al., 2020)). Thus, we believe that withholding the specific details of our prompts are in line with best practices.

Acknowledgments

We would like to acknowledge the Open Philanthropy project for funding this research in part.

References

Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

Maria Bada and Jason RC Nurse. The social and psychological impact of cyberattacks. In *Emerging cyber threats and cognitive vulnerabilities*, pp. 73–92. Elsevier, 2020.

Simon Bennetts. Owasp zed attack proxy. *AppSec USA*, 2013.

Daniil A Boiko, Robert MacKnight, and Gabe Gomes. Emergent autonomous scientific research capabilities of large language models. *arXiv preprint arXiv:2304.05332*, 2023.

Andres M Bran, Sam Cox, Oliver Schilter, Carlo Baldassari, Andrew White, and Philippe Schwaller. Augmenting large language models with chemistry tools. In *NeurIPS 2023 AI for Science Workshop*, 2023.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

Patrick Engebretson. *The basics of hacking and penetration testing: ethical hacking and penetration testing made easy*. Elsevier, 2013.

Richard Fang, Rohan Bindu, Akul Gupta, Qiusi Zhan, and Daniel Kang. Llm agents can autonomously hack websites, 2024.

Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. More than you've asked for: A comprehensive analysis of novel prompt injection threats to application-integrated large language models. *arXiv e-prints*, pp. arXiv–2302, 2023a.

Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, pp. 79–90, 2023b.

William G Halfond, Jeremy Viegas, Alessandro Orso, et al. A classification of sql-injection attacks and countermeasures. In *Proceedings of the IEEE international symposium on secure software engineering*, volume 1, pp. 13–15. IEEE Piscataway, NJ, 2006.